

Object Oriented Programming Concepts

- class
- object
- encapsulation
- interface
- inheritance
- abstraction
- polymorphism

```
class Main{
    public static void main(String[] a){
        Integer x = 10;
        System.out.println(a);
    }
}
class Test{
    Integer a = 20;
}
```

A.java

compilation: javac A.java

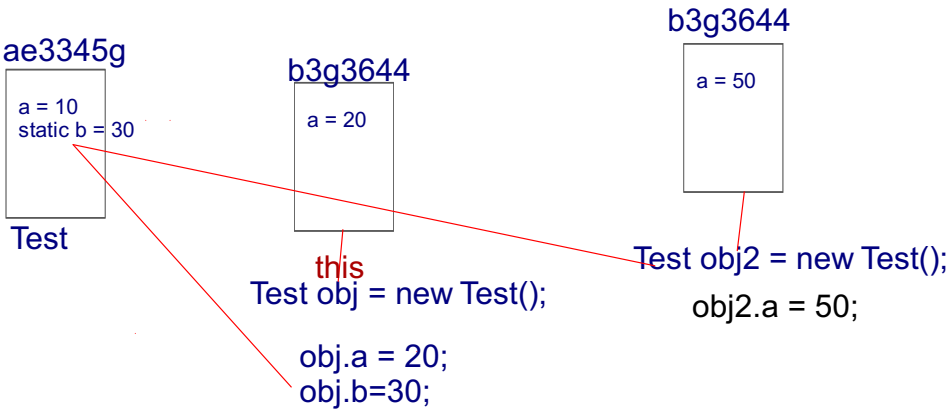


Main.class, Test.class (Byte Code)

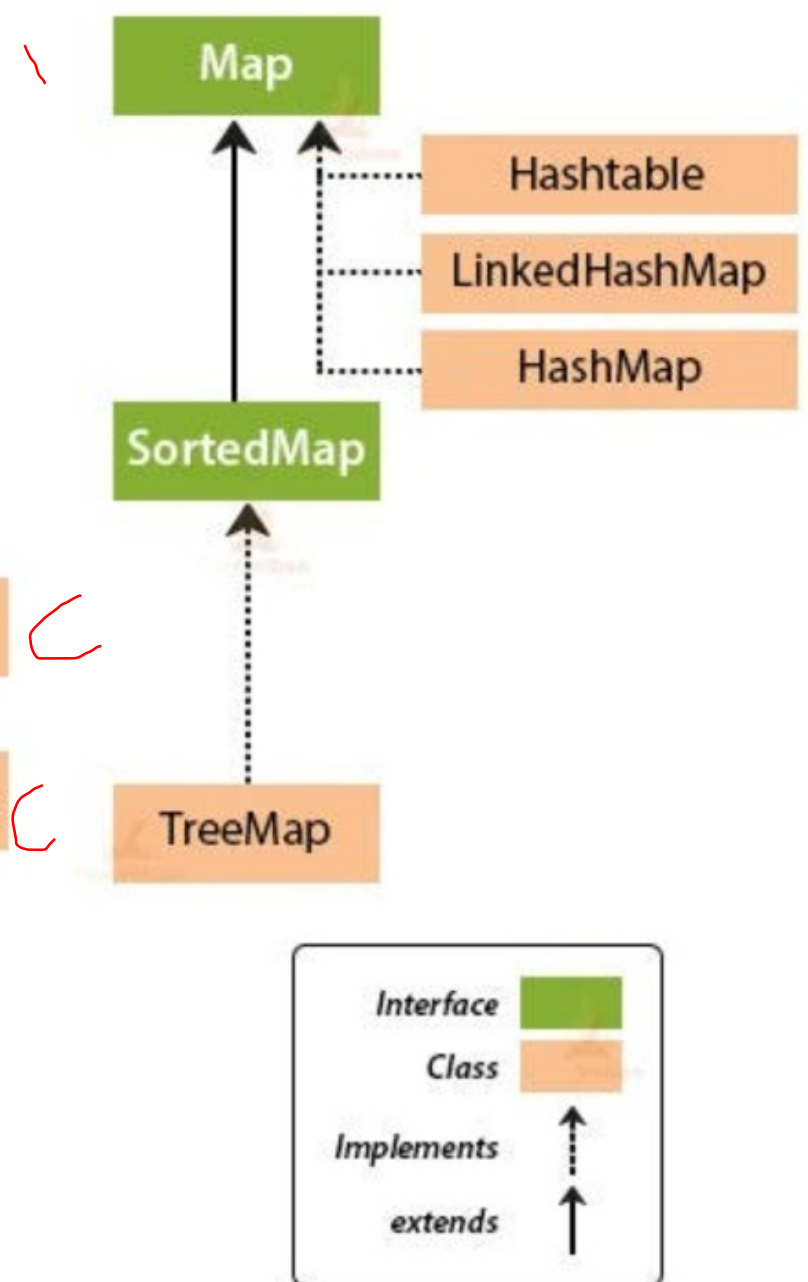
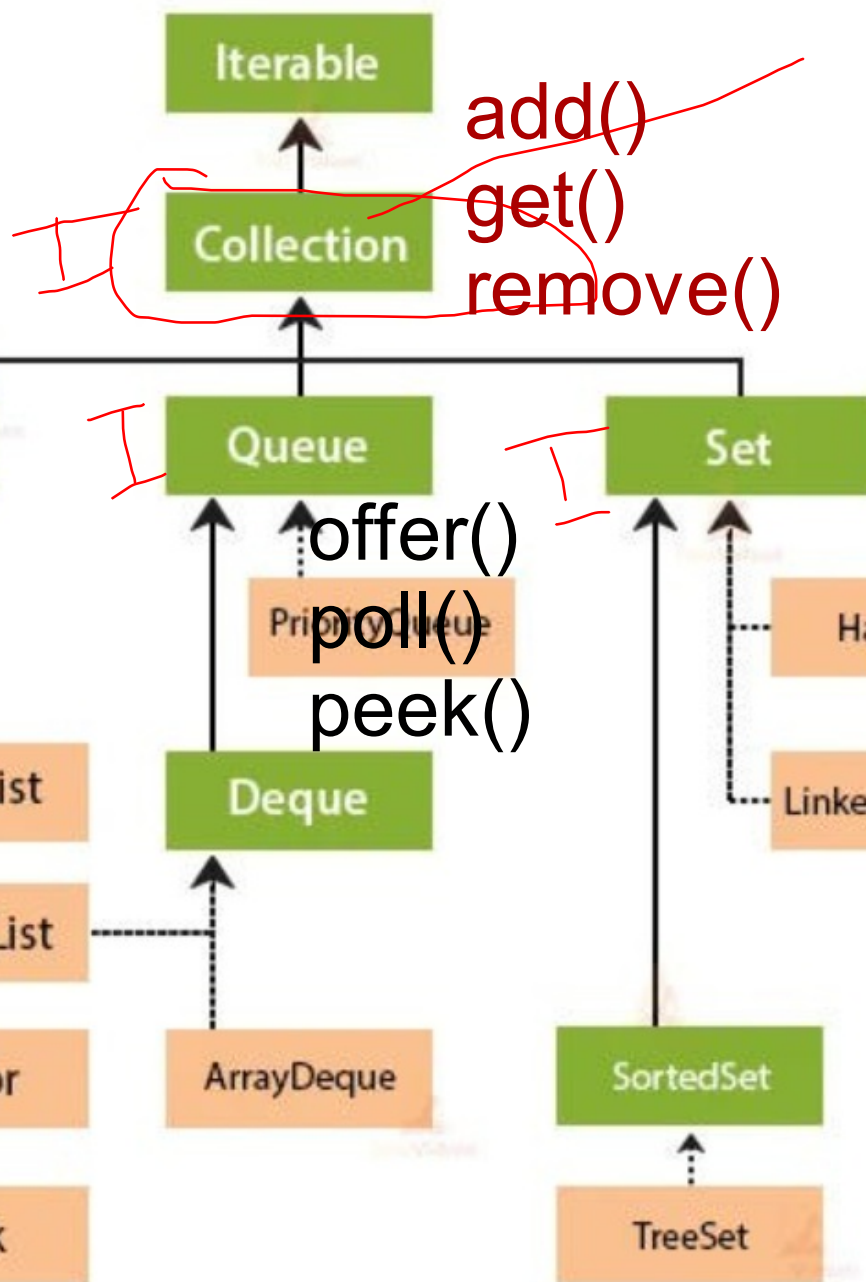
interpretation: java Main



Output



add(int index)
addFirst();
addLast();
get(index);
remove(index);
push();
pop();
peek();



UPI

Card

IB

COD

payment(amount)

Payment Modes

Packages

com.bit.model -> encapsulated classes

com.bit.service -> service interfaces

com.bit.dao -> service implementation classes

com.bit.controller -> Main class

Java Database Connectivity

Library
Java MySQL Connector



Prepare Statement
Execute Query
ResultSet
Close Connection

Naming Conventions

class/interface	PascalCase	Payment, PaymentService
field/method	camelCase	name, firstName, display(), readInput()
constants	UPPER_SNAKE_CASE	PIE, PERSONAL_LOAN_INTEREST
packages	lower case with dots	edu.bit.training

SOLID Principles

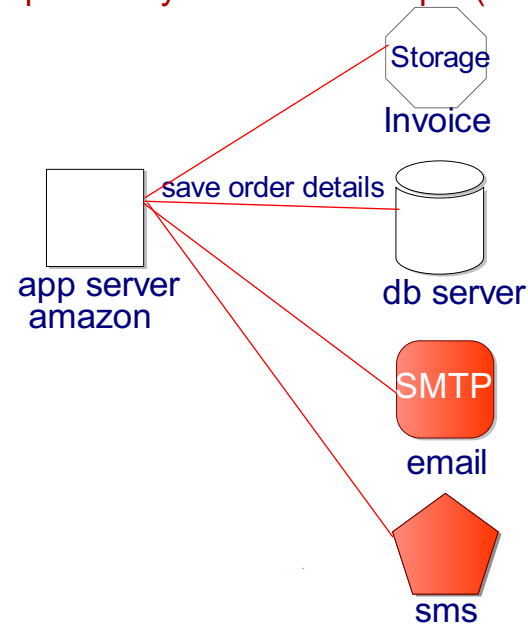
Single Responsibility Principle (SRP)

Open/Closed Principle (OCP)

Liskov Substitution Principle (LSP)

Interface Segregation Principle (ISP)

Dependency Inversion Principle (DIP)



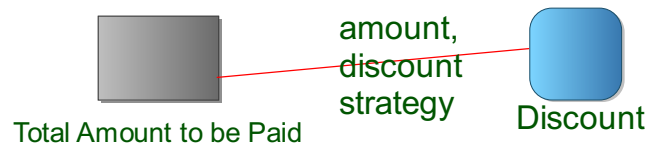
```
class OrderServiceImpl{
    //code to generate invoice
    //code to save in database
    //code to send mail with invoice
    //code to send sms with order info
}
```

After SOLID Principles - SRP

```
interface NotificationService{}
class NotificationServiceImpl implements NotificationService{}
interface InvoiceService{}
class InvoiceServiceImpl implements InvoiceService{}
interface OrderService{}
class OrderServiceImpl implements OrderService{
    //invoice service injection
    InvoiceService is = new InvoiceServiceImpl();
    NotificationService ns = new NotificationServiceImpl();
    //code to save order into database
}
```

Open/Closed Principle (OCP)

- open for extension and closed for modification



```
public interface DiscountStrategy {
    public Float calculateOrderAmount(Float amount, String discountMode);
}
public class DiscountStrategyImpl implements DiscountStrategy {
    @Override
    public Float calculateOrderAmount(Float amount, String discountMode) {
        if(discountMode == null) {
            return amount;
        }else if(discountMode.equals("RepublicSale")) {
            return (float) (amount - (amount * 0.1));
        }else if(discountMode.equals("DiwaliSale")) {
            return (float) (amount - (amount * 0.2));
        }
        return null;
    }
}
```

with out OCP, code needs to change everytime a new discount strategy needs to be added.

After OCP (Strategy Pattern)

```
public interface DiscountStrategy {
    public Float calculateOrderAmount(Float amount);
}
public class NoDiscountStrategy implements DiscountStrategy {
    @Override
    public Float calculateOrderAmount(Float amount) {
        return amount;
    }
}
public class RepublicSaleStrategy implements DiscountStrategy {
    @Override
    public Float calculateOrderAmount(Float amount) {
        return (float) (amount - (amount * 0.1));
    }
}
public class DiscountStrategyImpl implements DiscountStrategy {
    private DiscountStrategy discountStrategy;
    public DiscountStrategyImpl(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }
    public Float processOrderAmount(Float amount) {
        return this.discountStrategy.calculateOrderAmount(amount);
    }
}
```

```
//dynamic implementation of discount strategy
DiscountStrategyImpl discountStrategyImpl = new
DiscountStrategyImpl(new DiwaliSaleStrategy());

System.out.println("Total Amount to be paid: " +
discountStrategyImpl.processOrderAmount(23555.25f));
```

Liskov Sustitution Principle (LSP)

- Validation of input
- Exception Handling (custom exception for child classes)

```
public class DiscountException extends Exception {  
    public DiscountException(String message) {  
        super(message);  
    }  
}  
  
public class MainController {  
    public static void main(String[] args) throws DiscountException {  
        DiscountStrategyImpl discountStrategyImpl = new  
DiscountStrategyImpl(new DiwaliSaleStrategy());  
        Float amount = -23555.25f;  
        if(amount > 0) {  
            System.out.println("Total Amount to be paid: " +  
discountStrategyImpl.processOrderAmount(amount));  
        }else {  
            throw new DiscountException("Invalid amount...");  
        }  
    }  
}
```

Interface Segregation Principle (ISP) - reduces coupling

- have separate small interfaces instead of one big fat interface

```
public interface NotificationService {  
    public void communicate(String message);  
    public void communicateWithAttachment(String message);  
}  
  
public class SMSNotification implements NotificationService {  
    @Override  
    public void communicate(String message) { }  
    @Override  
    public void communicateWithAttachment(String message){ }  
}  
  
public class EmailNotification implements NotificationService {  
    @Override  
    public void communicate(String message) {}  
    @Override  
    public void communicateWithAttachment(String message){ }  
}
```

SMSNotification forced to implement
communicateWithAttachment() which is not relevant.

After ISP,

```
public interface NotificationService {  
    public void communicate(String message);  
}  
  
public interface EmailNotificationService{  
    public void communicateWithAttachment(String message);  
}  
  
public class EmailNotification implements NotificationService,  
EmailNotificationService { }  
  
public class SMSNotification implements NotificationService { }
```

Dependency Inversion Principle (DIP)

- High level modules should depend on abstraction, but not on concrete classes

```
public class DiscountStrategyImpl {  
    private DiscountStrategy discountStrategy;  
  
    //Dependency Injection  
    public DiscountStrategyImpl(DiscountStrategy discountStrategy) {  
        this.discountStrategy = discountStrategy;  
    }  
  
    public Float processOrderAmount(Float amount) {  
        return this.discountStrategy.calculateOrderAmount(amount);  
    }  
}
```

-> injecting different discount service classes based on a condition to calculate order amount

Singleton Pattern

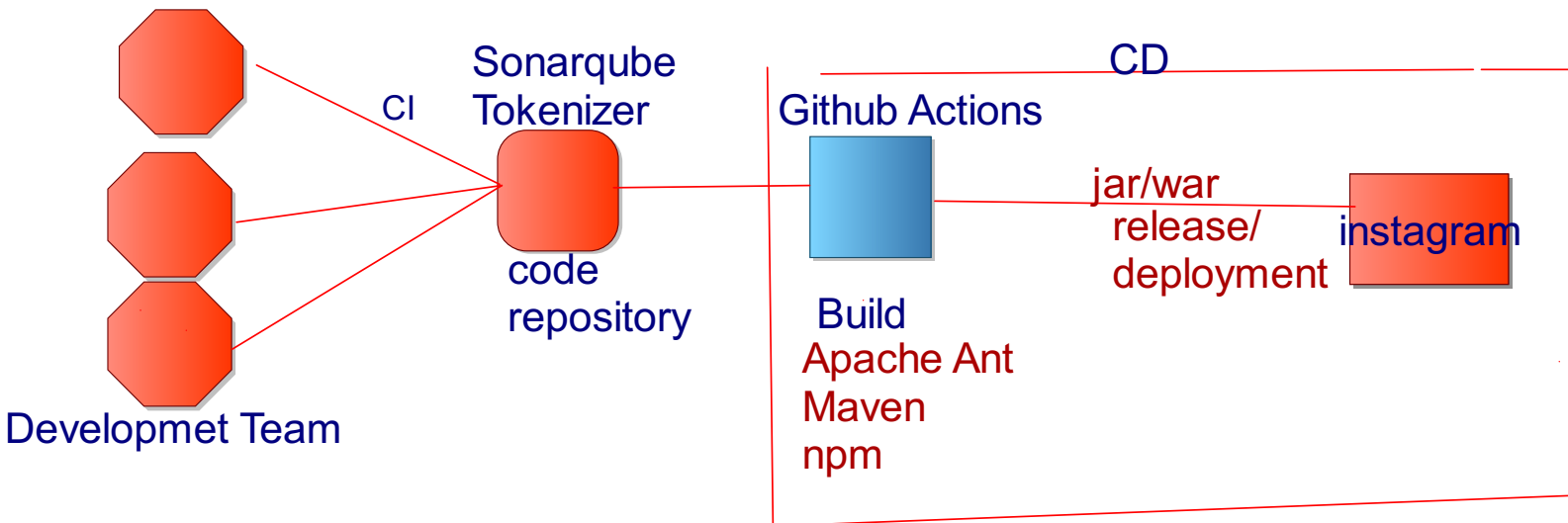
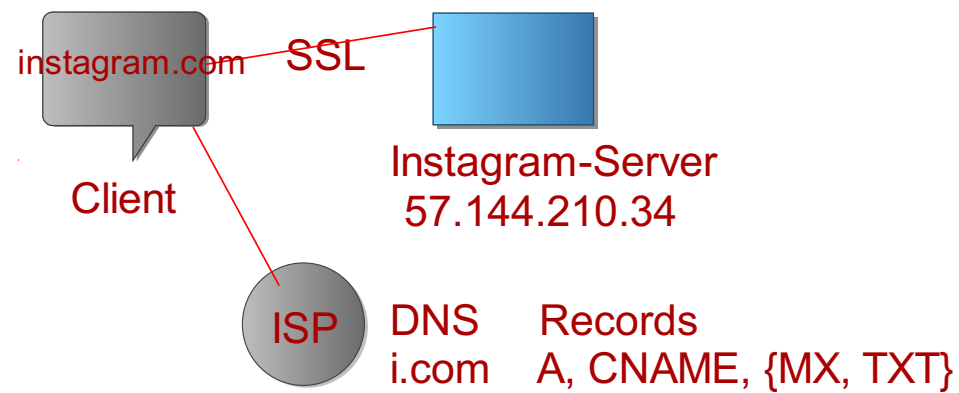
//Singleton class

```
public class MySqlManager {
    private static Connection connection = null;
    private static final String HOST_NAME = "jdbc:mysql://localhost:3306/ifi";
    private static final String USER_NAME = "root";
    private static final String PASSWORD = "Duck@!4#";
    private MySqlManager() {
        // private constructor to restrict object creation
    } //restricting external object creation
    public static Connection getMySqlConnection() {
        try {
            if(connection == null) {
                connection = DriverManager.getConnection(HOST_NAME,
USER_NAME, PASSWORD);
            }
        } catch (SQLException e) {
            System.err.println("MySQL connection failed...");
        }
        return connection;
    }
} //saves memory, global access, shared memory, and ensures consistency
```

Application Types

- Web
- Mobile
- Console/System/Stand-alone
- POS (Point of sales)
- IoT (Sensor data processing)
- AI/ML (LLM/SLM)
- Networking Apps

Client - Server Architecture



Application Architectures

- Monolithic
- n-tier Architecture
- Micro Service

